Lab 9: Milestones

Start Assignment

- Due Tuesday by 5pm
- Points 60
- Submitting a file upload

Lab 9: Milestones

Three milestones. Bring the sensor up, find out what its clock is really doing, then make it produce frames and prove it. Demo all three to the TA.

> Upload your code before you demonstrate.
>
> Your three files go to **this assignment**, and they must be showing as submitted in Canvas **before** the TA starts your demo.
>
> A demonstration with no code submitted scores **zero** for the milestone, whatever the board did.

> This lab will not finish inside the session, and it is not meant to
>
> There are three modules to write and four separate measurements to take. A pair who get through Milestone 1 and the clock readings in the room are on schedule.
>
> The lab is open all week. Work in simulation until everything passes, then build once — the package is arranged so you never need the board to make progress on the logic.

Milestone 1: The control plane (20 points)

Write `CMV300_Ctrl.v` and `CMV300_Spi.v`. The port lists are fixed and the testbench checks them.

The power-up sequencer

1. CLK_IN running, then at least 1 µs, then release SYS_RES_N, then at least 1 µs, then allow SPI.
2. A **sensor_ready** status bit the host can read, asserted only when the sequence has completed.
3. A FRAME_REQ pulse generator: one trigger in, a pulse of at least four CLK_IN periods out.

The SPI master

1. 16-bit frames, MSB first, at 10 MHz or below.

2. Drive SPI_IN so the sensor can sample it on its rising edge; sample SPI_OUT on the edge you justified in pre lab question 1(b).
3. Reads and writes, with the read data presented on a WireOut.

Acceptance

Check	Expected
Register defaults read back	reg 1 = 232, reg 2 = 1, reg 42 = 232, reg 61 = 85, reg 83 = 155
Number_lines reconstructed	232 + 256 × 1 = **488**
Write then read	write 100 to reg 1, read back 100
Reset restores defaults	sensor reset, then reg 1 reads 232 again
Temperature registers	regs 78 and 79 give roughly 5,000 to 5,700, rising as the sensor warms
Scope the bus	one SPI transaction captured on TP1–TP6, matching the datasheet figure

Why the composite is the real known answer

Reading 232 from reg 1 proves your master can move a byte. Reconstructing 488 from two registers proves the byte order and the bit alignment are right as well.

And 488 is a number you can check against the sensor itself: it is the number of rows.

Milestone 2: The clock, and the errata (12 points)

The given CLK_OUT meter counts sensor clock edges in exactly one millisecond and reports the count on a WireOut. Read it at three points, in order, from one script.

State	Meter reads	Why
After power-up, before reg 69 = 9	**0**	CLK_OUT is not routed to its pin yet
reg 69 = 9, reg 83 at its default 155	about **36,700**	the PLL locked in the 32–50 MHz band

reg 69 = 9, reg 83 = 251	**25,000 ± 16**	the 10–32 MHz band, matching your 25 MHz CLK_IN

State the percentage error the wrong band introduces into every subsequent timing measurement, and be ready to say where the ±16 spread comes from.

> This is the whole reason the meter exists
>
> With reg 83 at its datasheet default the sensor runs, responds to SPI, produces frames, and yields perfectly reasonable-looking images.
>
> Every timing number you derive from them is wrong by 47%, and there is no symptom anywhere except this counter. Be ready to explain to the TA why nothing else on the board complains.

Milestone 3: FRAME_REQ, in both regimes (28 points)

Write `CMV300_Counters.v`: the LVAL rising-edge counter, the DVAL-high cycle counter within a row, and the running bitwise OR and AND of every pixel of a frame.

3a. One frame, from idle

Configure Number_frames = 1, pulse FRAME_REQ once, and measure with the given frame-timing meter:

Quantity	Expected	What sets it
FRAME_REQ to the first LVAL	**6.55 ms**	exposure + FOT
First LVAL to the last LVAL	**12.69 ms**	the readout of 488 rows
LVAL rising edges	**488**	Number_lines
DVAL-high cycles per LVAL	**648**, as 324 + one invalid slot + 324	the row structure

> Your pixel counter will probably be wrong the first time
>
> A row is **not** 648 back-to-back valid pixels. If your counter assumes it is, it will still produce a number, and that number will be confidently wrong.

Fix it in simulation, where you can see exactly which cycle it went astray. If you generated this module, read what you were given before you run it: the obvious implementation is the wrong one, and the testbench will tell you so by naming the slot it went wrong at.

3b. Pipelined, and the exposure law

Now set Number_frames large and let the sensor run. Measure the frame-to-frame period. At the default exposure it should be **12.88 ms**, not the 19.24 ms of 3a. Report both and give the ratio.

Then measure the period at **three exposures**: one well below the knee, one near it, and one well above. Compare each with the prediction you brought, and report the exposure at which the period stops being flat.

3c. The five preconditions

Break the sensor on purpose, three ways, and record what the counters say for each:

1. Clear reg 69 and pulse FRAME_REQ.
2. Set Number_frames to 0 and pulse FRAME_REQ.
3. Pulse FRAME_REQ less than 1 ms after the SPI upload.

For each, report which counter revealed it and what its reading was.

3d. All ten data lines

Your OR/AND accumulator runs across every pixel of a frame. With any real scene in front of the sensor, the OR should have all ten bits set and the AND should be zero. Report both masks in hex, at a short exposure and a long one.

> Twenty flip-flops prove the parallel bus is alive without storing a single pixel.

Demo & Submission

Upload **three separate files** to this assignment, with exactly these names.

Filename	Contents
`lab9_CMV300_Ctrl.v`	your power-up sequencer and FRAME_REQ pulse generator
`lab9_CMV300_Spi.v`	your SPI master

`lab9_CMV300_Counters.v`	your row and pixel counters and the OR/AND masks

Your post lab PDF goes to the separate **Lab 9 – Post Lab** assignment.

The rules, in full

Situation	Result
Code uploaded, then demonstrated	graded normally on the checks below
Files missing or misnamed, noticed at the demo	fix it there and then. The TA waits. No penalty.
No code submitted by the deadline	**milestone scores 0**, regardless of the demonstration
Demonstration and upload both late	10% per hour, applied to the milestone as a whole

Due: First 60 minutes of your next lab session, the same deadline as the post lab.

What the TA Checks

#	The TA does this	This must happen
0	**Opens this assignment in Canvas before starting**	**all three code files are submitted and correctly named.**
1	Runs `python build.py --sim`	`RESULT: ALL PASS`, with your three modules in place of the stubs
2	Watches the register defaults read back	all five, and Number_lines reconstructed as 488
3	Looks at your scope capture of one SPI transaction	TP1–TP6, with the control, address and data fields marked
4	Asks which edge you sample SPI_OUT on and why	a reason from the sensor's behaviour, not from convention
5	Watches the three CLK_OUT readings in order	0, about 36,700, then 25,000 ± 16

6	Asks what the middle reading would have cost you	47% error on every timing number, with no other symptom
7	Watches one frame from idle	four quantities measured, each compared against a prediction
8	Watches the pipelined period, and the three exposures	12.88 ms at default, and the knee identified
9	Asks you to trigger one of the three deliberate failures	the counter that reveals it, named before the run
10	Looks at the OR and AND masks	OR all ten bits set, AND zero, at two exposures

Check 9 is the one that separates a working camera from an understood one. Name the counter first, then run it.